TALASH

Talent Acquisition & Learning Automation for Smart Hiring

Milestone 1 Report - Proposal, Architecture, Wireframes & Early Prototype

Course: CS 417 - Large Language Models (Spring 2026)
Program: BSDS-2023 | SEECS, NUST
Instructor: Prof. Dr. Muhammad Moazam Fraz

Group Members:
Mehreen Raheel – 460502
Eeshal Tahir – 465888
Huma Ejaz – 481480

1. Introduction

Recruitment is one of the most critical processes in any organization, yet it remains time-consuming, subjective, and heavily dependent on manual evaluation. Traditional hiring workflows rely on basic keyword matching and human judgment, often overlooking deeper aspects of a candidate's academic quality, research output, publication profile, experience consistency, and skill relevance.

TALASH (Talent Acquisition & Learning Automation for Smart Hiring) is an AI-powered recruitment support system focused on university hiring needs, built using Large Language Models (LLMs). The system is designed to automate CV screening, candidate-job matching, academic and publication analysis, experience validation, and skill alignment assessment - helping recruiters make more informed, evidence-based, and efficient hiring decisions.

This report documents Milestone 1 of the TALASH project, covering the system proposal, architecture design, UI/UX wireframes, and an early working prototype of the CV parsing pipeline.

2. Objectives and Scope

The primary objective of TALASH is to build an LLM-assisted recruitment system that performs comprehensive candidate evaluation beyond traditional keyword-based filtering. The specific goals for Milestone 1 are:

- Design the overall system architecture and module interaction pipeline
- Develop a preprocessing module that parses candidate CVs in PDF format
- Extract structured information including personal details, education, experience, skills, and publications
- Build an interactive Streamlit-based web prototype demonstrating the upload and parsing flow
- Design UI/UX wireframes for the planned full system
- Establish a folder-based CV ingestion pipeline for batch processing

The broader scope of TALASH includes - to be completed in subsequent milestones - educational profile scoring, research profile analysis (journal rankings, conference tiers, authorship roles), professional experience timeline validation, co-author network analysis, and a graphical dashboard for candidate comparison.

3. System Architecture

The TALASH system is organized into modular layers, each responsible for a discrete stage of the candidate evaluation pipeline. The architecture is designed for extensibility - each module can be developed and tested independently, and new analysis modules can be plugged in without disrupting existing ones.

3.1 High-Level Architecture

The high-level architecture consists of five primary layers:

- Input Layer - CV PDFs are ingested either through direct file upload or folder-based batch monitoring.
- Preprocessing Layer - Raw PDFs are read using pdfplumber, and text is passed to the parser module.

- Analysis Layer - The parser extracts structured fields (education, experience, publications, skills) using section-aware regular expressions tuned for the NUST HR portal CV format.
- Storage Layer - Parsed profiles are stored as structured JSON files, designed to feed into a relational database schema in future milestones.
- Presentation Layer - A Streamlit web application presents parsed results, flags missing information, and exposes raw extracted text and JSON output.

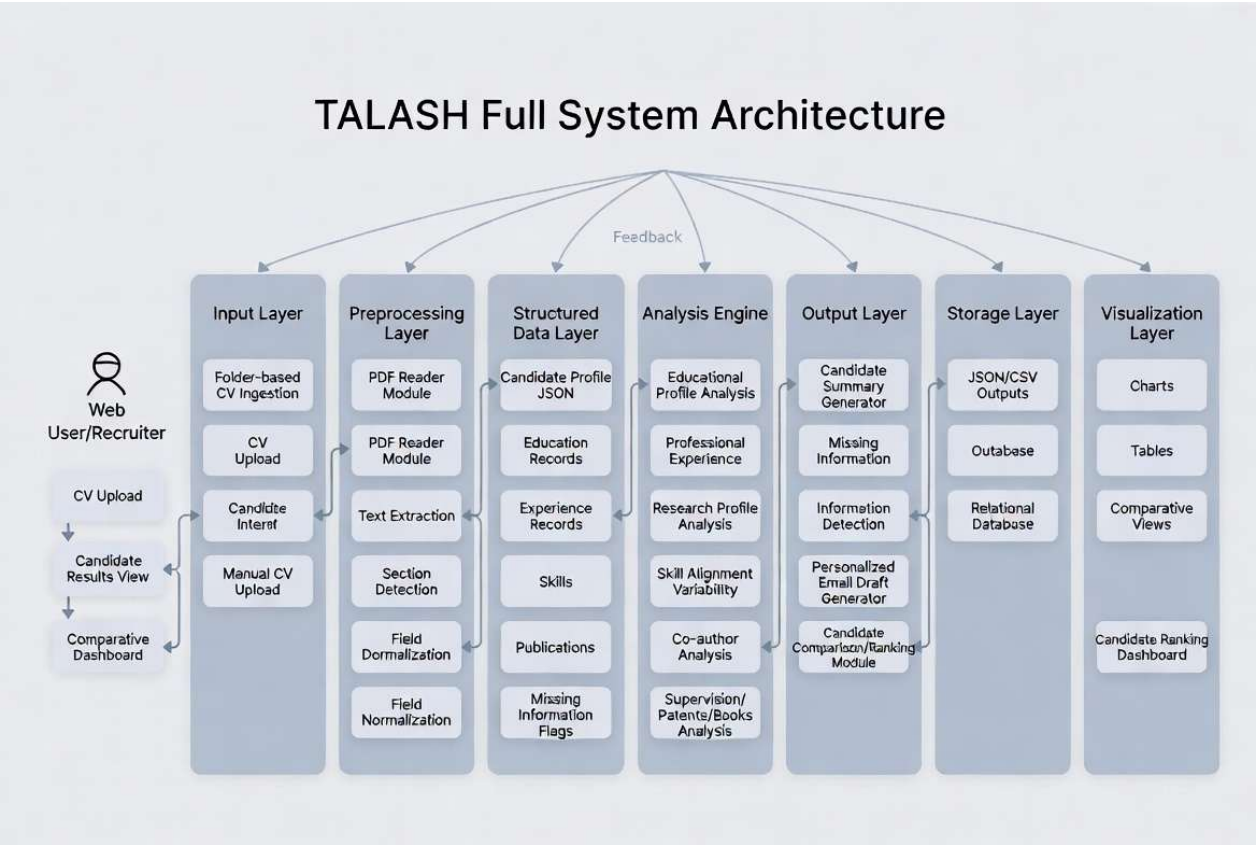

Figure 1: High-Level System Architecture

3.2 Detailed Architecture Diagram

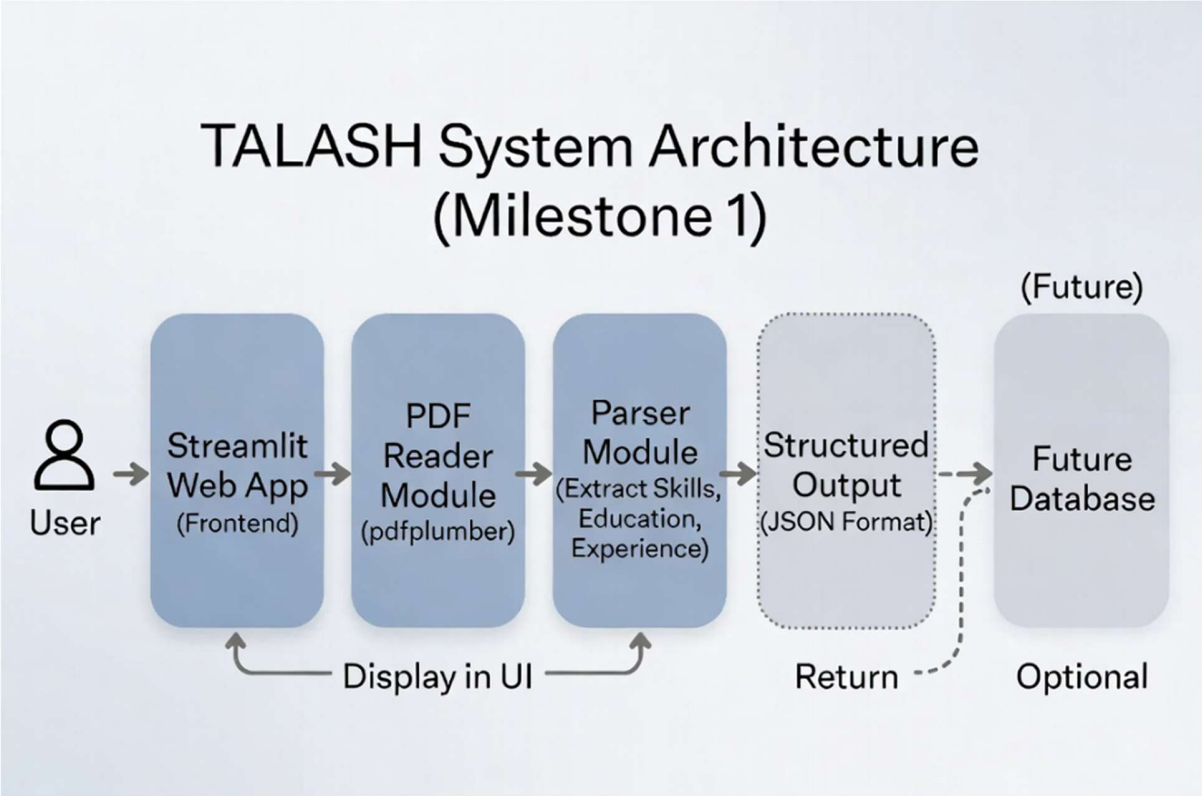

Figure 2: Detailed Architecture Diagram showing module interactions

3.3 Data Flow

The data flow through the TALASH system follows a linear pipeline from raw PDF input to structured JSON output, as illustrated below:

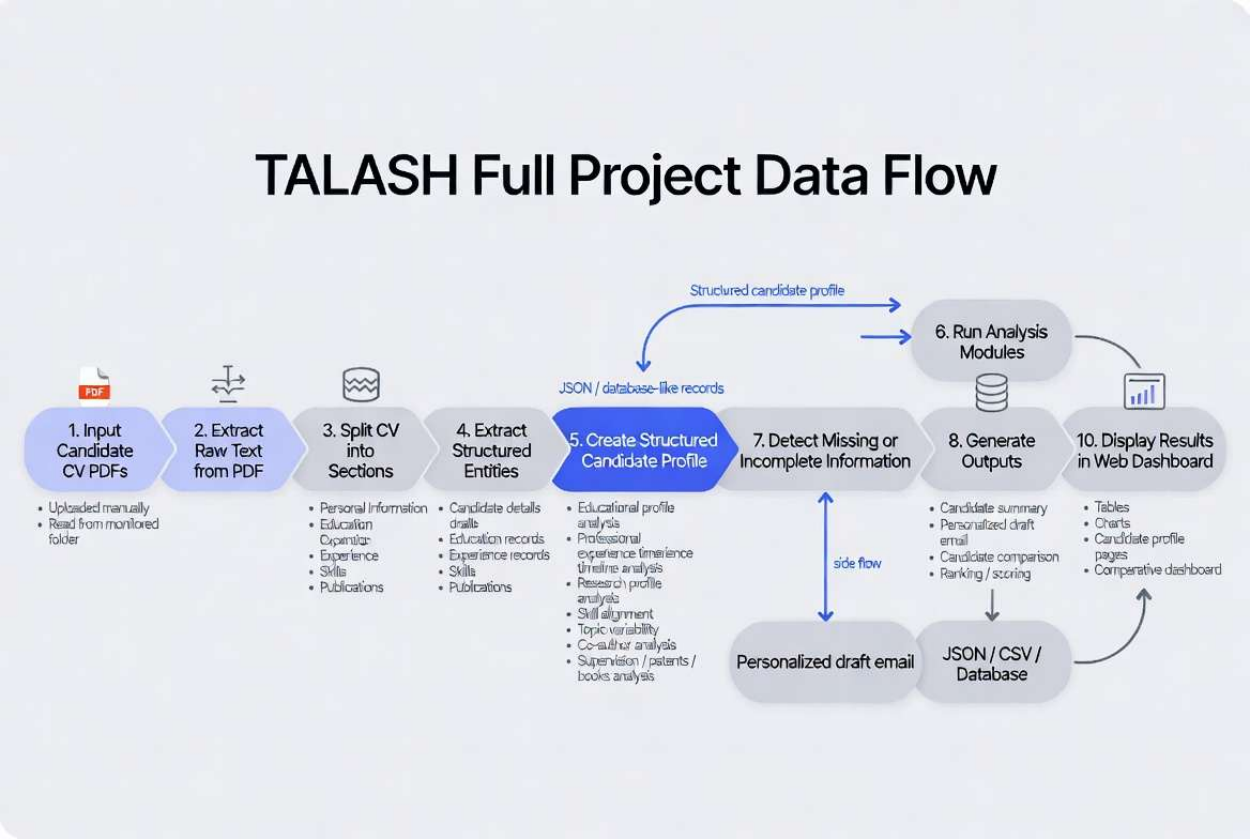

Figure 3: Full Data Flow Diagram

Figure 4: Detailed Data Flow Diagram

3.4 Module Descriptions

pdf_reader.py

Responsible for opening a PDF file using pdfplumber, iterating through all pages, extracting text with layout awareness, and returning the concatenated full-text string. Handles edge cases where page text extraction returns None.

parser.py

The core parsing module. It implements a section-aware pipeline that:

- Splits the raw CV text into named sections (header, education, experience, publications, skills, references) by detecting section header keywords
- Extracts personal information (name, email, phone, DOB, position applied for) from the header section using targeted regex patterns
- Parses education records from the space-aligned table format used by the NUST HR portal, extracting degree title, grade/CGPA, passing year, and institution
- Parses professional experience entries with job title, organization, and employment duration
- Extracts publication entries identifying journal vs. conference type and impact factor
- Performs a keyword-based skill scan across both the dedicated skills section and the full document text
- Detects and flags missing information fields for recruiter review

streamlit_app.py

The web interface module is built with Streamlit. It provides two input modes - direct PDF upload and folder-based CV selection - and renders all parsed profile fields including personal information, education, experience, publications, skills, missing information flags, raw extracted text, and formatted JSON output.

4. Database and Storage Design

For Milestone 1, parsed outputs are stored as structured JSON files in the data/sample_outputs/ directory. This serves as the intermediate data store and is designed to map cleanly onto a relational database schema planned for Milestone 2.

The JSON schema for each candidate's profile is structured as follows:

Top-Level Key	Data Type	Description
personal_information	Object	Name, email, phone, DOB, position applied for
education	Array of Objects	Degree, grade/CGPA, passing year, institution
experience	Array of Objects	Job title, organization, start year, end year, duration
publications	Array of Objects	Title, venue, type (Journal/Conference), year, impact factor
skills	Array of Strings	Detected technical and domain skills
missing_information	Array of Strings	Flags for absent or undetected fields

Table 1: JSON Output Schema

In Milestone 2, this schema will be mapped to a relational database with linked tables for candidates, education records, experience records, and publication records - enabling SQL-based querying, ranking, and comparison across multiple candidates.

5. UI/UX Wireframes and Design Thinking

Wireframes were designed for three core screens of the planned TALASH web application. The design prioritizes clarity, recruiter efficiency, and structured information presentation.

5.1 Upload Screen

The upload screen is the entry point. It provides a clean drag-and-drop or file browser interface for CV submission, with clear indication of accepted file types and upload status.

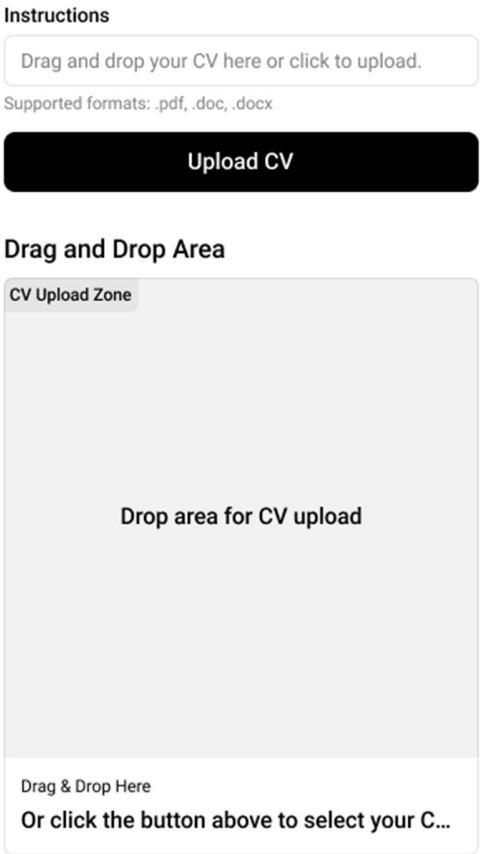

Figure 5: Wireframe - CV Upload Screen

5.2 Results Screen

The results screen displays the parsed candidate profile in a structured layout with sections for personal information, education, experience, skills, and publications. Missing information is flagged in a dedicated panel.

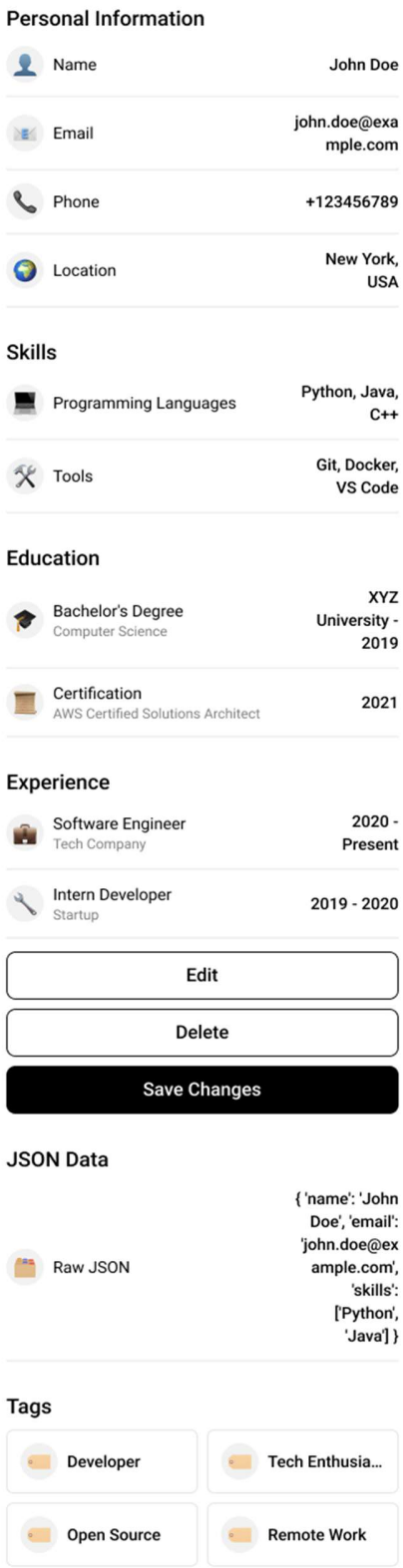

Figure 6: Wireframe - Parsed Results Screen

5.3 Dashboard Screen

The dashboard (planned for Milestone 2 and 3) provides graphical visualizations including publication quality breakdowns, skill alignment charts, educational progression timelines, and candidate comparison tables.

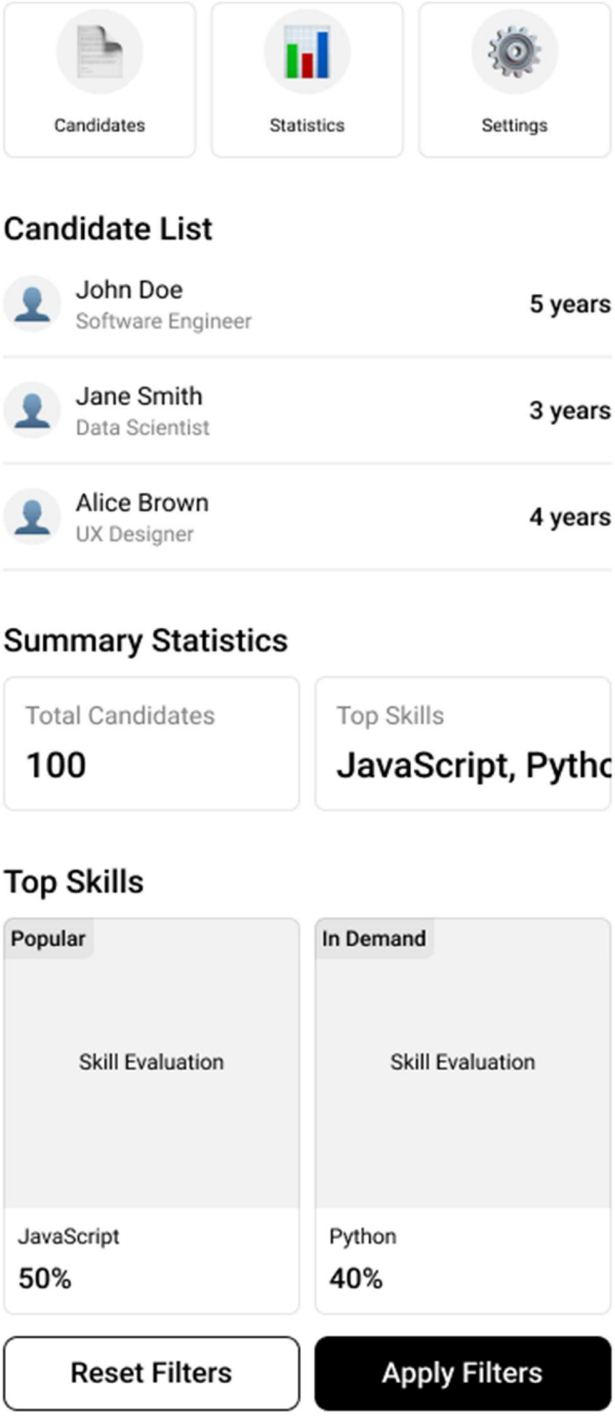

Figure 7: Wireframe - Analytics Dashboard

6. Preprocessing Module - Implementation

The preprocessing module is the central deliverable of Milestone 1. It converts unstructured CV text into a clean, machine-readable JSON structure suitable for downstream analysis.

6.1 PDF Text Extraction

PDF text extraction is handled by pdfplumber, chosen for its layout-aware extraction capabilities which preserve the spatial structure of space-aligned tables - the dominant format used in the NUST HR portal CVs. The extractor iterates page by page and concatenates output, handling pages that return no text gracefully.

6.2 Section Detection

The parser identifies section boundaries by scanning each line for known section header keywords (e.g., 'Education', 'Civil Experience', 'Publications', 'References'). Lines are routed to their respective section buckets, creating a dictionary of named text blocks that the downstream extractors operate on.

This approach is robust to variations in page breaks - a section that starts on one page and continues on the next is handled correctly because the full document text is processed as one stream before sectioning.

6.3 Field Extraction

Each field group uses a targeted extraction strategy:

- Personal Information - Regex patterns locate the candidate's name from the 'Name ... Father's' line pattern, the date of birth from 'Date/Place of Birth', and email/phone from pre-reference-section text (to avoid picking up referee contact details).
- Education - Lines containing degree keywords (PhD, MS, BSc, HSSC, SSC, etc.) are parsed with regex to extract grade values, four-digit passing years, and trailing institution names from the space-aligned table layout.
- Experience - Lines matching job title keywords (Professor, Lecturer, Engineer, etc.) are parsed using a month-year duration regex (e.g., 'Sep-2017 - Aug-2023') to extract start and end dates, with title and organization separated by keyword boundary detection.
- Publications - A state-machine approach accumulates title lines until a Journal/Conference venue keyword is encountered, at which point the buffered title and venue metadata (impact factor, year, type) are flushed as a complete entry.
- Skills - A two-pass scan checks the dedicated skills section first, then falls back to a full-text keyword scan against a curated list of 40+ technical and domain skills relevant to ECE and CS faculty recruitment.
- Missing Information - A structured checker flags absent email, phone, DOB, education, experience, and publications, as well as missing grade fields within detected education entries.

6.4 Sample Parsed Output

Below is a representative parsed output for candidate Muhammad Salman (PhD, Electrical Engineering - Communication Systems):

Field	Extracted Value
Name	Muhammad Salman
Position Applied	Assistant Professor - ECE, SEECS
PhD	Electrical Engineering, CGPA 3.33/4.0 - IIU Islamabad (2024)
MS	Electrical Engineering, CGPA 3.03/4.0 - COMSATS Islamabad (2014)
BSc	Electronics Engineering, 69.84% - COMSATS Abbottabad (2011)
HSSC	Pre-Engineering, 65.45% - B.I.S.E Bannu (2006)
SSC	Science, 71% - B.I.S.E Bannu (2004)
Experience	5 roles: Support Eng → Program Coord → Lecturer → Asst. Prof → Assoc. Prof (2012–Present)
Publications	5 entries: 4 Journal (IF 2.10–2.80), 1 Conference
Skills Detected	Deep Learning, Optimization, Research
Missing Flags	Email, Phone, DOB (not printed on portal-generated CVs)

Table 2: Sample Parsed Output - Muhammad Salman

> Users > Hp > Downloads > Talash-main > Talash-main > data > sample_outputs > {} Muhammad_Salman.json > ...


```
2   "personal_information": {
3     "name": "Muhammad Salman",
4     "email": "",
5     "phone": "",
6     "position_applied": "Assistant Professor - Electrical & Computer Engineering - SEECS",
7     "dob": ""
8   },
9   "education": [
10    {
11      "degree": "PhD in Electrical Engineering Communication Systems",
12      "grade": "3.33",
13      "year": "2024",
14      "institution": "International Islamic university, Islamabad"
15    },
16    {
17      "degree": "MS in Electrical Engineering Communication Systems",
18      "grade": "3.03",
19      "year": "2014",
20      "institution": "COMSATS University, Islamabad, Attock Campus"
21    },
22    {
23      "degree": "BSc Electronics Engineering Electronics",
24      "grade": "69.84",
25      "year": "2011",
26      "institution": "COMSATS University, Islamabad, Abbottabad Campus"
27    },
28    {
29      "degree": "HSSC Pre-Engineering",
30      "grade": "65.45",
31      "year": "2006",
32      "institution": "B.I.S.E Bannu"
33    },
34    {
35      "degree": "SSc Science Subjects",
36      "grade": "71",
37      "year": "2004",
38      "institution": "B.I.S.E Bannu"
```

7. Early Prototype - Screenshots

The Streamlit prototype was developed and tested with real candidate CVs sourced from the NUST HR recruitment portal. The following screenshots demonstrate the full working flow.

7.1 Application Home Screen

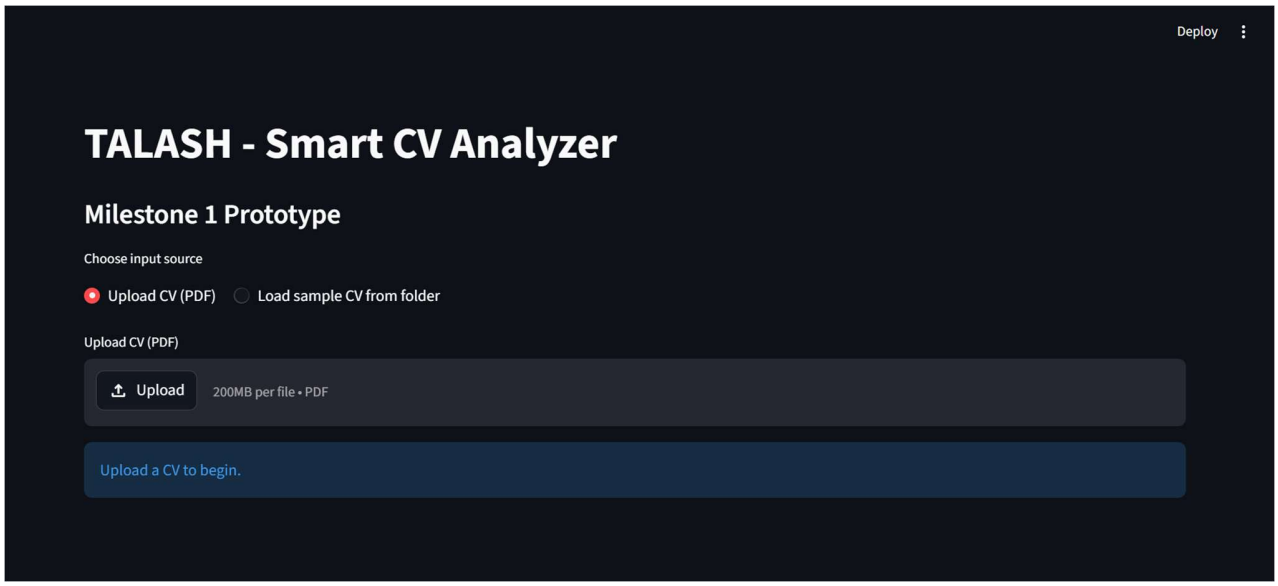

Figure 8: Home Screen - two input modes available (Upload PDF / Load from folder)

7.2 CV Upload and Processing

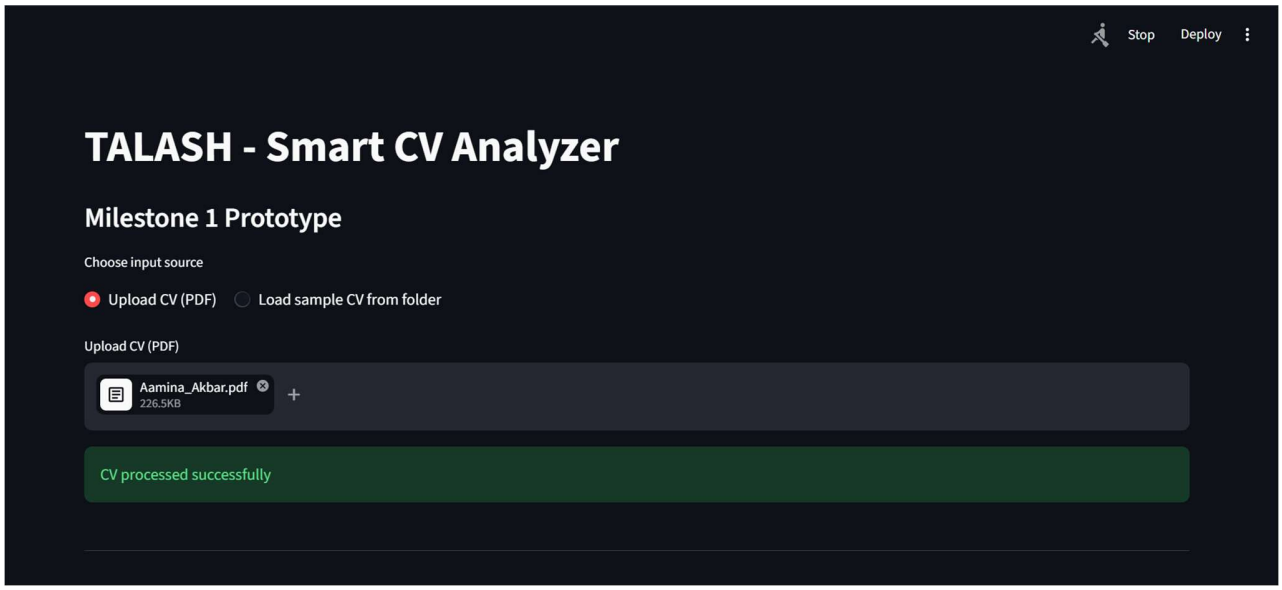

Figure 9: CV successfully uploaded and processed

7.3 Parsed Profile Output - Personal Info and Skills

Figure 10: Parsed personal information and detected skills

7.4 Parsed Profile Output - Education and Experience

Figure 11: Extracted education records and professional experience

7.5 Parsed Profile Output - Publications and Missing Flags

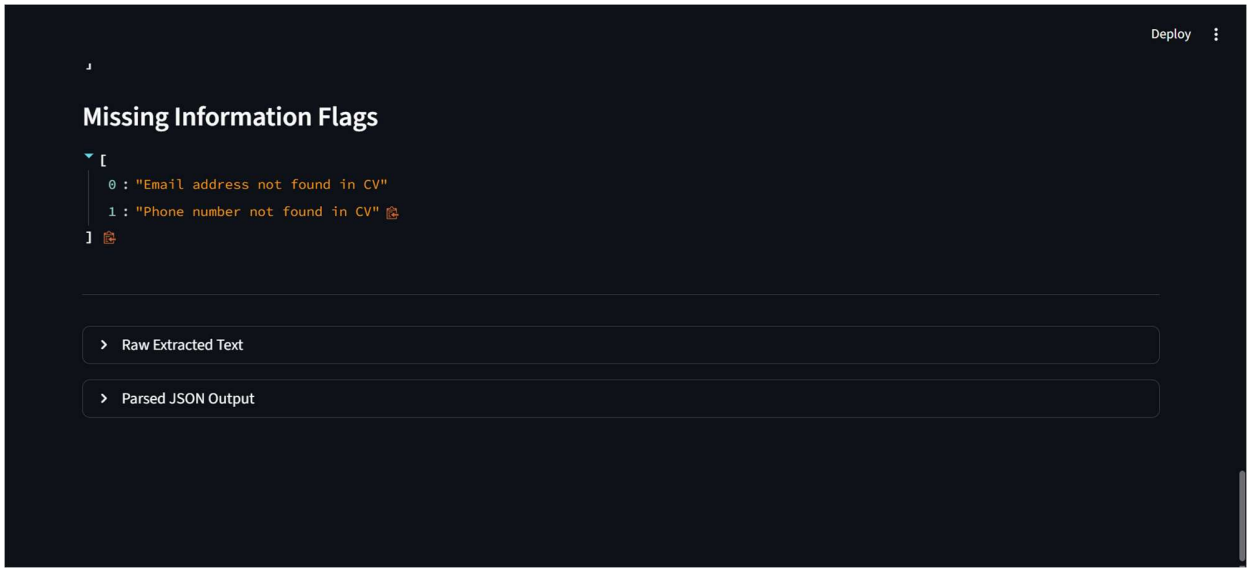

Figure 12: Extracted publications and missing information flags

7.6 Raw Extracted Text

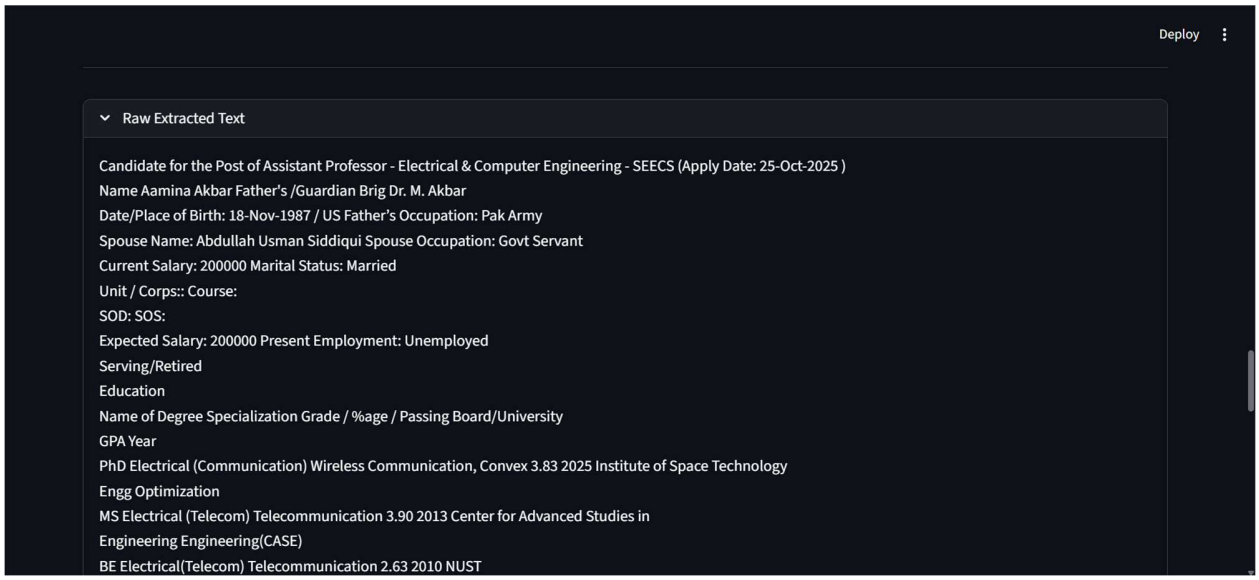

Figure 13: Raw text extracted from CV PDF (expandable panel)

7.7 JSON Output

Figure 14: Full structured JSON output for the parsed candidate

7.8 Folder-Based Input Mode

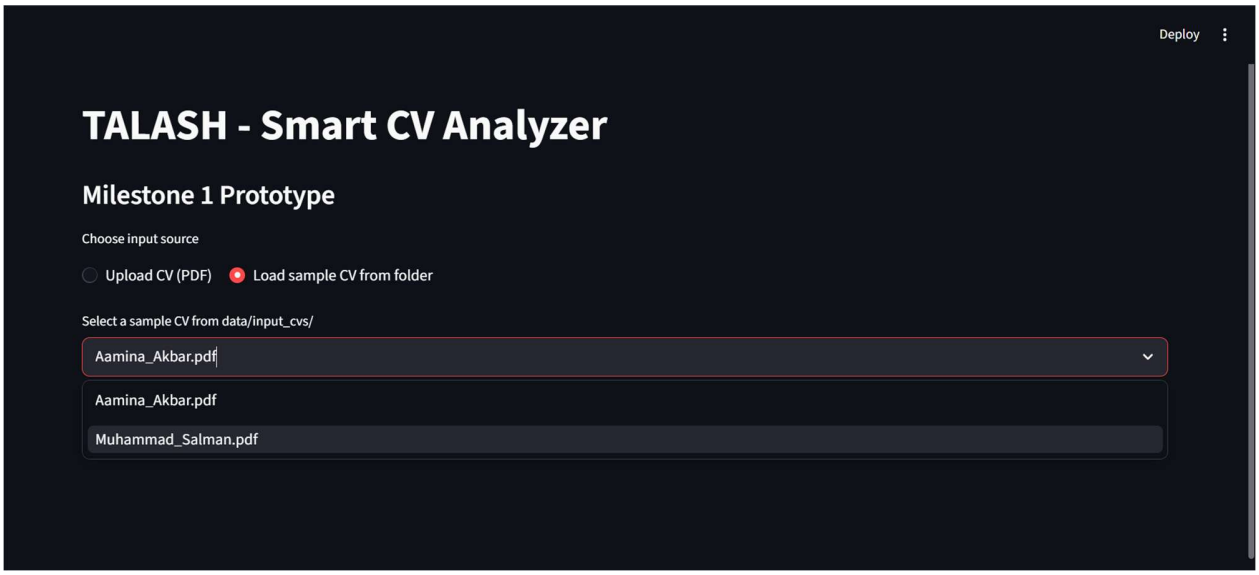

Figure 15: Folder-based input mode - selecting from pre-loaded CVs in data/input_cvs/

8. Technologies Used

Technology	Version	Purpose
Python	3.14	Primary programming language
Streamlit	≥ 1.33.0	Web application framework and UI
pdfplumber	≥ 0.11.0	Layout-aware PDF text extraction
pandas	≥ 2.2.0	Tabular data handling and status display
python-dateutil	≥ 2.9.0	Date parsing utilities
re (stdlib)	Built-in	Regular expression-based field extraction
json (stdlib)	Built-in	JSON serialization of parsed profiles
pathlib (stdlib)	Built-in	File system path handling
Git / GitHub	-	Version control and collaboration

Table 3: Technologies and Libraries

9. Repository Structure

The GitHub repository is organized as follows:

Path	Description
app/streamlit_app.py	Main Streamlit web application
preprocessing/pdf_reader.py	PDF text extraction using pdfplumber
preprocessing/parser.py	Section-aware CV parsing pipeline
data/input_cvs/	Sample CV PDFs for folder-based input mode
data/sample_outputs/	Pre-parsed JSON profiles for demo mode
docs/	Architecture diagrams, wireframes, screenshots
docs/screenshots/	Prototype screenshots for all app screens
requirements.txt	Python dependency specifications
README.md	Setup and run instructions

Table 4: Repository Structure

10. Milestone 1 Completion Status

Deliverable	Status
System architecture and technical design	Complete
UI/UX wireframes and design thinking	Complete
Preprocessing module - PDF reader	Complete

Deliverable	Status
Preprocessing module - CV parser	Complete
Early prototype with upload/read flow	Working
Folder-based CV ingestion design	Implemented
Preliminary extraction from real CVs (2)	Complete
Missing information detection	Implemented
LLM/NLP pipeline design (planned)	Milestone 2
Advanced analysis modules	Milestones 2–3

Table 5: Milestone 1 Completion Checklist

11. Limitations

11.1 Current Limitations

- The parser is tuned for the NUST HR portal CV format. CVs with significantly different layouts (e.g., free-form academic CVs) may require additional parsing rules.
- Publication title extraction picks up some co-author name fragments due to the multi-column space-aligned layout of the publications table in these CVs.
- Email and phone are not printed on portal-generated CV PDFs - they are stored in the portal system. Missing flags are correctly raised for these fields.
- Skill detection relies on keyword matching; skills not in the curated list will not be detected.

12. Conclusion

Milestone 1 of TALASH has been successfully completed. The team has delivered a working preprocessing pipeline that accurately parses real candidate CVs from the NUST HR portal, a Streamlit-based web prototype with both upload and folder-based input modes, a complete system architecture design, UI/UX wireframes, and structured JSON output with missing information detection.

The foundation laid in this milestone - section-aware CV parsing, structured JSON profiles, and an interactive prototype - provides a solid base for the LLM-powered analysis modules planned for Milestones 2 and 3. The system is actively version-controlled on GitHub with a continuous commit history reflecting iterative development by both team members.